

Objetivos da aula:

- Introduzir o conceito de combinação linear e sua interpretação geométrica e computacional.
- Apresentar o span como conjunto gerador e relacioná-lo à construção de subespaços vetoriais.
- Desenvolver a noção de dependência e independência linear como critérios de redundância entre vetores.
- Introduzir o conceito de base como conjunto mínimo gerador e discutir sua relevância para representação de dados.
- Apresentar a ideia de coordenadas em uma base e a resolução de sistemas para obtê-las.
- Introduzir formalmente a mudança de base e sua interpretação geométrica e computacional.
- Realizar práticas iniciais em Python/NumPy para visualizar spans, verificar independência e resolver sistemas de coordenadas.

Índice:

- I. Motivação computacional
- II. Combinação Linear
- III. Conjunto Gerador (Span)
- IV. Dependência e Independência Linear
- V. Bases Vetoriais
- VI. Coordenadas em uma Base
- VII. Mudança de Base

i. Motivação computacional

A Álgebra Linear é a estrutura fundamental sobre a qual todo o Aprendizado de Máquina (ML) é construído. Para um cientista de dados ou engenheiro de ML, entender vetores e combinações lineares não é uma necessidade acadêmica, mas uma ferramenta prática de depuração e modelagem.

Todo modelo de IA opera, essencialmente, manipulando vetores através de duas operações principais: Soma e Multiplicação por Escalar.

1. O Dado como Vetor (Representação)

Antes de qualquer processamento, o mundo real precisa ser traduzido para a linguagem da máquina:

- Visão Computacional: Uma imagem não é vista como "foto", mas como um vetor de intensidades de pixels.
- NLP (Processamento de Linguagem Natural – disciplina do 6º semestre): Palavras e frases são convertidas em Embeddings (vetores densos). A "distância" e a "direção" entre esses vetores capturam o significado semântico (ex: a direção de "Rei" para "Rainha" é similar à de "Homem" para "Mulher").

2. Redução de Dimensionalidade e Espaços Latentes

Dados reais são ruidosos e têm dimensões excessivas. Modelos modernos tentam encontrar um **Subespaço** (uma versão simplificada do universo de dados) onde a informação essencial reside.

- **Sistemas de Recomendação:** Representam usuários e filmes em subespaços de menor dimensão (vetores latentes) para prever interesses.
- **Compressão (PCA - Análise de Componentes Principais):** Projeta os dados em uma nova **Base** que maximiza a variância e descarta redundâncias (dependência linear).

ii. Combinação Linear

A combinação linear é, matematicamente, a operação fundamental que une a **soma de vetores** e a **multiplicação por escalar** – as duas operações que definem um espaço vetorial.

Uma combinação linear nada mais é do que um modo de misturar vetores usando **pesos** (números reais). Esses pesos podem alongar, encolher ou inverter vetores, e o resultado final é um vetor obtido pela soma ponderada desses vetores.

Em ML e IA, esta não é apenas uma definição acadêmica; é o mecanismo central pelo qual os modelos processam e transformam dados.

1. Definição algébrica formal

Uma combinação linear de um conjunto de vetores $\{v_1, v_2, \dots, v_k\}$ é qualquer vetor que pode ser construído pela soma dos múltiplos escalares desses vetores.

$$v = \alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_k v_k$$

- **Vetores (v_i):** são os "ingredientes" ou as "direções de base". Em ML, podem ser os vetores de entrada (dados) ou as características (features) extraídas.
- **Escalares (α_i):** são os **pesos** ou **coeficientes**. Eles ditam a quantidade de cada vetor que será usado na mistura. Em ML, são os parâmetros (pesos) que a rede neural aprende.
- **Vetor Resultante (v):** é a "mistura final" ou o "novo vetor". Em ML, é o vetor de saída de uma camada neural antes da ativação.

2. Interpretação Geométrica: Criando deslocamentos

Geometricamente, uma combinação linear é uma maneira de alcançar qualquer ponto em um espaço vetorial, movendo-se ao longo das direções definidas pelos vetores v_i , com a magnitude (escala) definida pelos pesos α_i .

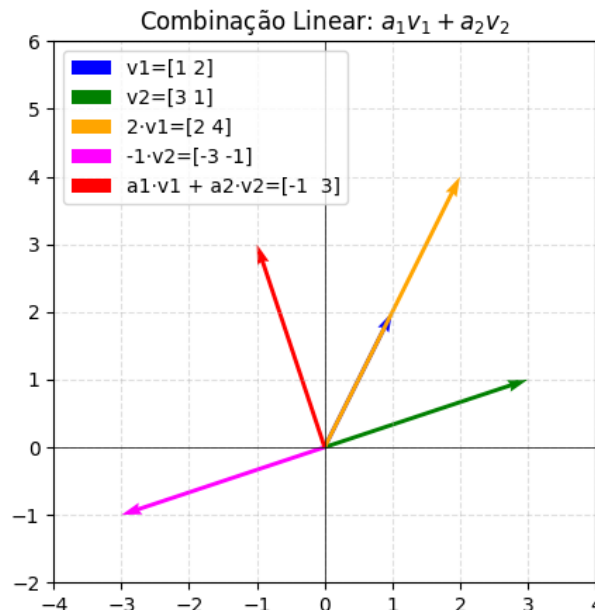
1. **Escalar o vetor:** Cada vetor v_i é alongado, encurtado ou invertido (multiplicação por escalar α_i).
2. **Somar os resultados:** Todos os vetores escalados são somados de ponta a ponta (soma de vetores), resultando no novo vetor v .

Exemplo em R^2 :

Considere $v_1 = (1,2)$ e $v_2 = (3,1)$. O vetor $v = 2v_1 - 1v_2$ é uma combinação linear, com $\alpha_1 = 2$ e $\alpha_2 = -1$.

$$v = \alpha_1 v_1 + \alpha_2 v_2 = 2(1,2) + (-1)(3,1) = (2,4) + (-3,-1) = (-1,3)$$

O ponto $(-1,3)$ (linha vermelha no gráfico) é alcançado com **duas unidades** na direção de v_1 (linha laranja no gráfico) e **uma unidade no sentido oposto** à direção de v_2 (linha magenta no gráfico).



iii. Conjunto Gerador (Span)

O conceito de Conjunto Gerador (Span) é a continuação natural da ideia de combinação linear.

- Combinação linear permite construir *um vetor específico*.
- Span descreve *todos os vetores* que conseguimos construir usando combinações lineares.

Em outras palavras:

O span mostra “quanto espaço” um conjunto de vetores consegue preencher.

1. Definição Formal

Dado um conjunto de vetores $\{v_1, v_2, \dots, v_k\}$ o span é definido como:

$$\text{span}\{v_1, \dots, v_k\} = \{\alpha_1 v_1 + \dots + \alpha_k v_k \mid \alpha_i \in \mathbb{R}\}$$

Interpretação simples:

O span é o conjunto de **todos** os pontos que podemos alcançar a partir da origem, usando apenas os vetores fornecidos e variando os pesos α_i .

2. Exemplo em Python

Uma maneira intuitiva de visualizar o span é desenhar todas as combinações lineares de dois vetores e mostrar o “espaço que eles preenchem”.

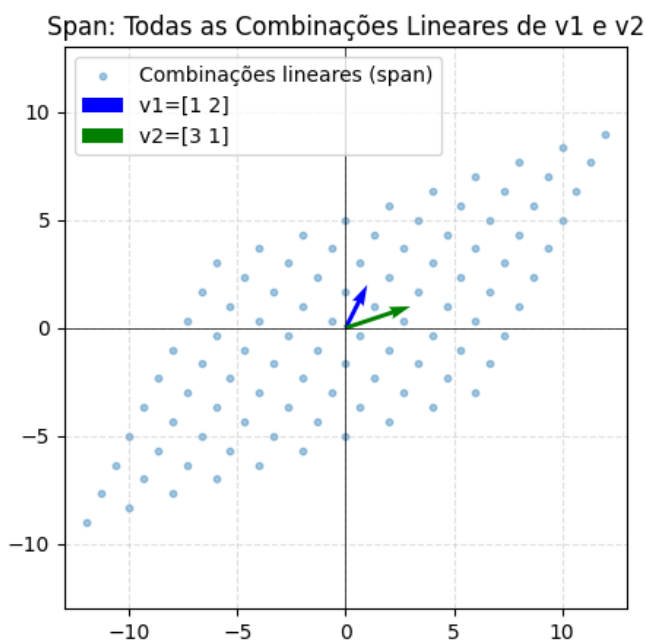
- Se v_1 e v_2 forem **colineares**, o span será **uma reta**.
- Se forem **independentes**, o span preencherá **uma região do plano**.

Computacionalmente, não podemos gerar “infinitos” pontos. O que o código a seguir faz é uma amostragem:

- Define um intervalo de escalares (pesos). A função `a_vals = np.linspace(-3, 3, 10)` gera 10 números (amostras) uniformemente espaçados entre -3 e 3;
- Usa loops aninhados para testar todas as misturas possíveis desses pesos.

```
for a1 in a_vals:
    for a2 in a_vals:
        ponto = a1 * v1 + a2 * v2
```

- Plota cada resultado como um ponto azul. A “nuvem” de pontos azuis aproxima visualmente o subespaço preenchido pelos vetores, mostrando se eles formam uma linha fina ou preenchem o plano.



Note que, embora o computador mostre apenas pontos isolados, matematicamente o span preenche **todo** o espaço entre esses pontos, formando um plano sólido e infinito.

Para que o gráfico se aproxime de algo mais parecido com um "plano preenchido", sugiro que você aumente de 10 para 25.

- Como está: 10 amostras → 100 pontos no gráfico (parece uma rede de pesca aberta).
- Sugestão: 25 amostras → 625 pontos no gráfico (parece um tecido fechado).

```
import numpy as np
import matplotlib.pyplot as plt

# Defina os vetores base
v1 = np.array([1, 2])
v2 = np.array([3, 1])

# Geração de combinações lineares
a_vals = np.linspace(-3, 3, 10) # Aumente de 10 para 25 e veja o resultado
pontos_span = []
# Calculando os pontos azuis do gráfico
for a1 in a_vals:
    for a2 in a_vals:
        ponto = a1 * v1 + a2 * v2
        pontos_span.append(ponto)

pontos_span = np.array(pontos_span)

# Plotagem
plt.figure(figsize=(5, 5))
```

```
plt.scatter(pontos_span[:, 0], pontos_span[:, 1],
            s=10, alpha=0.4, label='Combinações lineares (span)')

plt.quiver(0, 0, v1[0], v1[1],
           angles='xy', scale_units='xy', scale=1,
           color='blue', label=f'v1={v1}')

plt.quiver(0, 0, v2[0], v2[1],
           angles='xy', scale_units='xy', scale=1,
           color='green', label=f'v2={v2}')

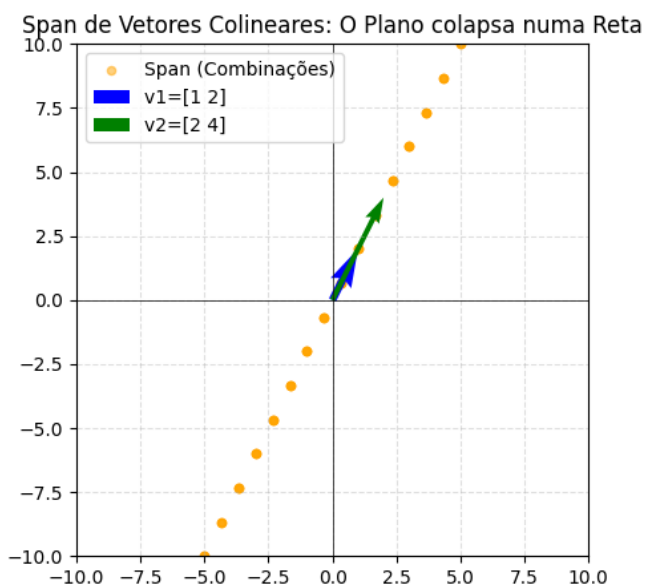
lim = np.max(np.abs(pontos_span)) + 1
plt.xlim(-lim, lim)
plt.ylim(-lim, lim)

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)

plt.grid(True, linestyle='--', alpha=0.4)
plt.gca().set_aspect('equal', adjustable='box')
plt.legend()
plt.title('Visualização do Span: Todas as Combinações Lineares de v1 e v2')

plt.show()
```

Exemplo de span de vetores colineares



1. **O colapso dimensional:** Ao rodar o código, vemos que, embora tenhamos dois vetores e dois loops de repetição variando os pesos, todos os pontos laranja caem perfeitamente sobre uma única linha diagonal.
2. **Redundância:** O vetor verde v_2 aponta exatamente na mesma direção que o azul v_1 , apenas é mais longo. Ele não oferece uma "nova direção" para explorar o plano R^2 .
3. **Conceito de span:** Isso prova visualmente que $\text{span}\{v_1, v_2\}$ é uma reta.

```
import numpy as np
import matplotlib.pyplot as plt
# 1. Definição de vetores COLINEARES (Redundantes)
# Para demonstrar o colapso de dimensão, v2 deve ser múltiplo de v1.
```

```
v1 = np.array([1, 2])
v2 = np.array([2, 4]) # v2 = 2 * v1 (estão na mesma linha)

# 2. Geração de combinações lineares (Amostragem)
# Mude para 20 para ver a linha claramente
a_vals = np.linspace(-3, 3, 10)
pontos_span = []

for a1 in a_vals:
    for a2 in a_vals:
        # A mágica acontece aqui: como v2 é redundante,
        # somar v1 e v2 não nos tira da direção original.
        ponto = a1 * v1 + a2 * v2
        pontos_span.append(ponto)

pontos_span = np.array(pontos_span)

# 3. Plotagem
plt.figure(figsize=(5, 5))
# Plotando os pontos gerados (Span)
plt.scatter(pontos_span[:, 0], pontos_span[:, 1],
            s=20, alpha=0.5, color='orange', label='Span (Combinações)')
# Plotando os vetores originais
plt.quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1,
            color='blue', label=f'v1={v1}', width=0.015)
plt.quiver(0, 0, v2[0], v2[1], angles='xy', scale_units='xy', scale=1,
            color='green', label=f'v2={v2}', width=0.010)
# Configurações visuais
lim = 10
plt.xlim(-lim, lim)
plt.ylim(-lim, lim)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True, linestyle='--', alpha=0.4)
plt.gca().set_aspect('equal', adjustable='box')
plt.legend()
plt.title('Span de vetores colineares: O Plano colapsa numa Reta')
plt.show()
```

3. Conexão com Subespaços

O span é sempre um subespaço porque:

- contém o vetor nulo,
- é fechado sob soma,
- é fechado sob multiplicação por escalar.

iv. Dependência e Independência Linear

Agora que entendemos o span (o espaço que podemos alcançar), surge uma pergunta fundamental para a computação:

"Precisamos mesmo de todos esses vetores para cobrir esse espaço, ou algum deles é inútil?"

É aqui que entram os conceitos de Dependência e Independência Linear. Eles medem a **redundância** da informação.

1. Intuição: Redundância de Direções

Imagine os vetores como *direções de movimento*.

- Se podemos nos mover para **norte** e para **leste**, já conseguimos chegar a qualquer ponto do plano.
- Se alguém acrescenta o movimento **nordeste**, isso não muda o espaço que podemos alcançar.
 - Nordeste = 1·Norte + 1·Leste
 - Não há novo "sentido de deslocamento".

Nesse caso, dizemos que o vetor "Nordeste" é Linearmente Dependente (LD). Ele é redundante e não aumenta o Span.

Por outro lado, se o terceiro movimento fosse "**subir**" (saindo do mapa plano), ganhariamos uma nova dimensão (altura). Esse vetor seria Linearmente Independente (LI).

2. Definição Formal

Um conjunto de vetores é:

- **Linearmente Dependente (LD):** Se pelo menos um dos vetores pode ser escrito como uma combinação linear dos outros. Isso significa que ele não traz "novas direções" para o span;
- **Linearmente Independente (LI):** Se **nenhum** vetor pode ser formado pelos outros. Cada vetor traz uma dimensão nova e exclusiva ao conjunto.

Um conjunto de vetores $\{v_1, v_2, \dots, v_k\}$ é LD se existe uma combinação linear não trivial que resulta no vetor nulo:

$$a_1v_1 + a_2v_2 + \dots + a_kv_k = 0$$

com **pelo menos um coeficiente não nulo**.

Se a única maneira de obter o vetor nulo é:

$$a_1 = a_2 = \dots = a_k = 0$$

então os vetores são LI.

3. Critério Prático (o teste do zero)

Para verificar independência:

- monte a equação

$$a_1v_1 + a_2v_2 + \dots + a_kv_k = 0$$

- se a única solução for $a_1 = a_2 = \dots = 0$, o conjunto é LI;
- se houver solução não trivial, é LD.

No NumPy, usamos `np.linalg.matrix_rank` para verificar isso rapidamente.

4. Exemplos Simples

Exemplo 1 – Dependência

$$v_1 = (1,2), \quad v_2 = (2,4)$$

Observe que $v_2 = 2v_1$, logo, são LD.

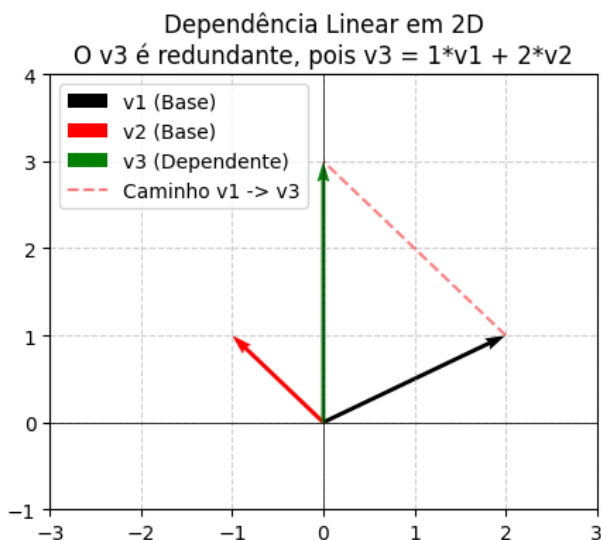
Exemplo 2 – Independência

$$v_1 = (1,0), \quad v_2 = (0,1)$$

Nenhum é múltiplo do outro.

Eles geram $\mathbb{R}^2$, logo, são LI.

5. Visualização em Python: O Teorema da Dimensionalidade



Observe que o vetor v_3 não aponta para "fora" da tela ou para uma direção impossível. Ele é perfeitamente alcançável combinando os vetores v_1 e v_2 . Isso prova visualmente que v_1, v_2, v_3 é um conjunto LD. Se quiséssemos descrever qualquer ponto desse plano, v_1 e v_2 seriam suficientes; v_3 é informação repetida.

```
import numpy as np
import matplotlib.pyplot as plt

# Vetores Base (LI entre si)
v1 = np.array([2, 1])
v2 = np.array([-1, 1])
# O vetor v3 é criado propositalmente como combinação de v1 e v2
v3 = 1 * v1 + 2 * v2      # Vetor Verde (Redundante)

origin = [0], [0] # cria uma tupla contendo duas listas

plt.figure(figsize=(5, 4))

# Plotar os vetores
plt.quiver(*origin, v1[0], v1[1], color='black', scale=1, scale_units='xy',
          angles='xy', label='v1 (Base)')
```

```
plt.quiver(*origin, v2[0], v2[1], color='red', scale=1, scale_units='xy',
angles='xy', label='v2 (Base)')
plt.quiver(*origin, v3[0], v3[1], color='green', scale=1, scale_units='xy',
angles='xy', label='v3 (Dependente)')

# Mostra que v3 é alcançável andando 1x em v1 e depois 2x em v2
plt.plot([v1[0], v3[0]], [v1[1], v3[1]], 'r--', alpha=0.5, label='Caminho v1 ->
v3')

# Ajustes do gráfico
plt.xlim(-3, 3)
plt.ylim(-1, 4)
plt.grid(True, linestyle='--', alpha=0.6)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend(loc='upper left')
plt.title("Dependência Linear em 2D\nv3 é redundante, pois v3 = 1*v1 + 2*v2")
plt.show()
```

6. Importância na Computação

- Compressão de Dados: Se temos 100 vetores, mas 20 são LD, podemos descartá-los sem perder nenhuma informação sobre o espaço gerado. Isso é a base da compressão (como em JPEG ou MP3).
- ML (feature selection): Se duas colunas (features) do nosso banco de dados são linearmente dependentes (ex: "idade em anos" e "idade em meses"), o modelo matemático pode falhar ou ficar instável, pois há informação duplicada. Isso é chamado de **multicolinearidade**.
- Sistemas Lineares: Um sistema só tem solução única se as equações forem independentes.

v. Bases Vetoriais

Após compreender combinação linear, span e independência linear, estamos preparados para chegar ao conceito fundamental que organiza todo espaço vetorial: a **Base**.

Se o span indica *o alcance* de um conjunto de vetores e a independência linear indica *o quanto esses vetores trazem direções novas*, então a base é o **ponto de equilíbrio perfeito** entre os dois:

uma base é o menor conjunto possível que ainda é capaz de gerar todo o espaço.

1. Definição: Por que precisamos de uma Base?

Pense que queremos descrever qualquer ponto do espaço usando apenas algumas direções principais. Por exemplo, no plano ($\mathbb{R}^2$):

- Com as direções “leste” e “norte”, conseguimos chegar a qualquer ponto.
- A direção “nordeste” não adiciona nada — é redundante (dependente).
- Com apenas “leste” e “norte”, **geramos o plano inteiro sem redundância**.

Esse conjunto de duas direções é uma **base do plano**.

Analogia: pense nas cores RGB.

- Para criar qualquer cor na tela, precisamos de vermelho, verde e azul.
- Esse conjunto R, G, B é uma Base das cores.
- Se tirarmos o Azul, não conseguimos mais fazer roxo ou ciano (o Span diminui).
- Se adicionarmos "amarelo", não ganhamos novas cores, pois Amarelo já é vermelho + verde. O conjunto $\{R, G, B, amarelo\}$ seria redundante (LD), logo, não é uma base eficiente.

2. Definição Formal

Um conjunto de vetores

$$\mathcal{B} = \{b_1, b_2, \dots, b_n\}$$

é uma **base de um espaço vetorial** V se satisfizer:

1. Gera o espaço (Span):

$$\text{span}(\mathcal{B}) = V$$

Ou seja, qualquer vetor de V pode ser escrito como combinação linear de $b_1, \dots, b_n$.

2. É LI: Nenhum vetor em $\mathcal{B}$ pode ser escrito como combinação linear dos outros.

3. A ideia central: Conjunto Mínimo Gerador

Uma base é exatamente isso:

- Se **removemos** um vetor da base, ela deixa de gerar o espaço inteiro.
- Se **adicionamos** um vetor, ele se torna redundante e o conjunto deixa de ser LI.

Resumo:

Uma base é o conjunto mínimo e suficiente de vetores que descreve todo o espaço.

4. Propriedade de Ouro: Representação Única

A característica mais importante de uma base para a computação é a **Unicidade**.

Se temos uma base, qualquer vetor do espaço pode ser escrito como uma combinação linear dos vetores da base de uma única maneira.

Isso significa que cada vetor tem um "endereço" ou "receita" único naquele sistema.

Esses pesos únicos da combinação chamamos de Coordenadas.

$$v = c_1 b_1 + c_2 b_2$$

Se mudarmos a base (os vetores b), as coordenadas (c) mudam, mas o vetor v (o ponto no espaço) continua o mesmo.

A maior vantagem prática de uma base é:

Cada vetor do espaço tem uma única representação como combinação linear dos vetores da base.

Exemplo:

Se $\mathcal{B} = \{b_1, b_2\}$ é uma base de $\mathbb{R}^2$, então existe um único par (a_1, a_2) tal que:

$$v = a_1 b_1 + a_2 b_2$$

Os números a_1 e a_2 são as **coordenadas** de v na base $\mathcal{B}$.

Este conceito será aprofundado no próximo capítulo.

5. Visualização em Python: Grades (Grids) de Coordenadas

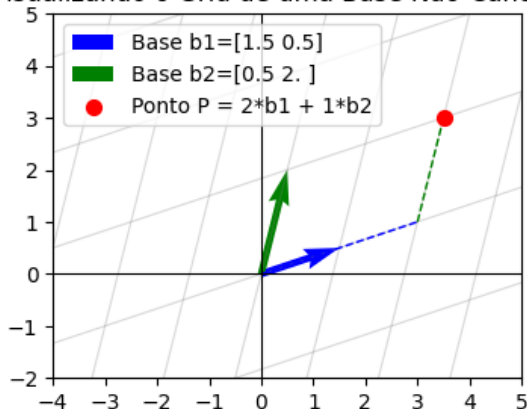
Visualmente, uma base define um "grid" ou uma malha sobre o espaço.

- Base Canônica (grid padrão) $\{[1,0], [0,1]\}$: linhas verticais e horizontais formam quadrados perfeitos.
- Base Não-Canônica (grid oblíquo): os quadrados se transformam em paralelogramos. Mas a cobertura do espaço continua perfeita.

Isso é fundamental para Computação:

- A mudança de base "distorce" a visão do espaço,
- mas mantém as relações lineares preservadas.

Visualizando o Grid de uma Base Não-Canônica



O que observar no gráfico:

- O **grid cinza** não é feito de quadrados perfeitos, mas de paralelogramos. Essa é a "visão de mundo" definida por essa base;
- Qualquer ponto onde as linhas cinzas se cruzam é uma combinação inteira da base;
- O ponto vermelho tem um "endereço" simples nesta base (2 passos no azul, 1 passo no verde), mesmo que suas coordenadas xy padrão pareçam números quebrados.

vi. Coordenadas em uma Base

Nos capítulos anteriores aprendemos que uma **base** é um conjunto mínimo de vetores que gera todo um espaço.

Agora veremos como **descrever vetores usando essa base**, ou seja, como encontrar suas **coordenadas**.

A ideia principal é simples:

Coordenadas são os "pesos" da combinação linear que reconstrói um vetor a partir da base.

1. Intuição: Um endereço relativo

Quando escrevemos um vetor como

$$v = (3,2)$$

na base canônica, isso significa:

- 3 passos na direção do eixo x (vetor $e_1 = (1,0)$)
- 2 passos na direção do eixo y (vetor $e_2 = (0,1)$)

Ou seja:

$$(3,2) = 3e_1 + 2e_2$$

As coordenadas **são justamente esses números 3 e 2**.

Mas se mudarmos a base, mudam as instruções — e **mudam as coordenadas**.

O ponto geométrico continua o mesmo, mas sua **descrição** muda de acordo com a base escolhida.

2. Definição Formal

Dada uma base

$$\mathcal{B} = \{b_1, b_2, \dots, b_n\}$$

de um espaço vetorial V , todo vetor $v \in V$ pode ser escrito de forma **única** como:

$$v = c_1b_1 + c_2b_2 + \dots + c_nb_n$$

Os escalares $c_1, c_2, \dots, c_n$ são chamados de

coordenadas de v na base $\mathcal{B}$

e são representados por

$$[v]_{\mathcal{B}} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}$$

3. Problema Computacional: Como encontrar as coordenadas?

Dado:

- o vetor v (em coordenadas padrão),
- e os vetores da base $\mathcal{B} = \{b_1, b_2\}$,

queremos descobrir os coeficientes c_1 e c_2 tais que

$$v = c_1b_1 + c_2b_2$$

Isso é equivalente a resolver o sistema linear:

$$\begin{bmatrix} b_1 & b_2 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = v$$

Ou seja:

Encontrar coordenadas é resolver um sistema linear.

No Python, fazemos isso com:

```
coords = np.linalg.solve(M_base, v)
```

onde `M_base` é a matriz com os vetores da base nas **colunas**.

4. Exemplo Numérico

Queremos representar o vetor

$$v = (4,3)$$

na base

$$b_1 = (2,1), \quad b_2 = (-1,1)$$

Montamos o sistema:

$$c_1 b_1 + c_2 b_2 = c_1(2,1) + c_2(-1,1) = (4,3)$$

Escrevendo como sistema de equações:

$$\begin{cases} 2c_1 - c_2 = 4 \\ c_1 + c_2 = 3 \end{cases}$$

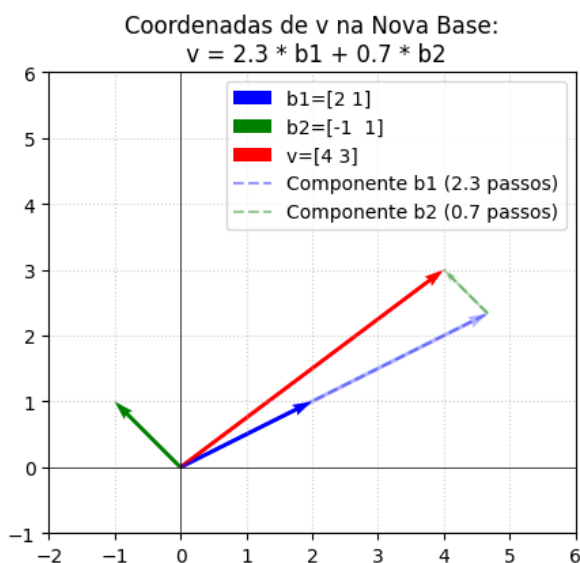
A solução é:

$$c_1 = 2, \quad c_2 = 1$$

Logo:

$$[v]_B = (2,1)$$

5. Visualização Geométrica



Explicação do Gráfico:

- O vetor v é o destino.
- Os vetores b_1 e b_2 são a nossa **base**.
- As linhas pontilhadas mostram o "caminho" seguindo as instruções da base.
- O código calcula exatamente quanto precisamos "esticar" o azul e o verde para chegar ao destino. Esses valores de esticamento (c_1, c_2) são as coordenadas.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Definindo o Vetor Alvo (v) e a Nova Base (b1, b2)
v = np.array([4, 3]) # Vetor alvo (em coordenadas padrão)

# Nova Base (Base B)
b1 = np.array([2, 1]) # Vetor Base 1
b2 = np.array([-1, 1]) # Vetor Base 2

# 2. Calculando as Coordenadas (Resolvendo o Sistema Linear)
```

```
# Montamos a Matriz de Transformação com b1 e b2 como COLUNAS
# Sistema: c1*b1 + c2*b2 = v --> Matriz * c = v
M_base = np.column_stack((b1, b2))

# Resolvemos para encontrar c (as coordenadas na nova base)
# c = [c1, c2]
coords = np.linalg.solve(M_base, v)
c1, c2 = coords

print(f"Vetor original (Padrão): {v}")
print(f"Novas Coordenadas na Base B: c1={c1:.2f}, c2={c2:.2f}")
print(f"Verificação: {c1:.2f}*{b1} + {c2:.2f}*{b2} = {c1*b1 + c2*b2}")

# 3. Visualização
origin = [0], [0]
plt.figure(figsize=(5, 5))

# Plotando a Base B
plt.quiver(*origin, b1[0], b1[1], color='blue', scale=1, scale_units='xy',
angles='xy', label=f'b1={b1}')
plt.quiver(*origin, b2[0], b2[1], color='green', scale=1, scale_units='xy',
angles='xy', label=f'b2={b2}')

# Plotando o vetor V
plt.quiver(*origin, v[0], v[1], color='red', scale=1, scale_units='xy',
angles='xy', label=f'v={v}')

# Visualizando a decomposição (O Caminho)
# Primeiro andamos c1 na direção de b1
step1 = c1 * b1
plt.quiver(*origin, step1[0], step1[1], color='blue', scale=1, scale_units='xy',
angles='xy', alpha=0.3)

# Depois, a partir de onde paramos, andamos c2 na direção de b2
plt.quiver(step1[0], step1[1], c2*b2[0], c2*b2[1], color='green', scale=1,
scale_units='xy', angles='xy', alpha=0.3, width=0.005)

# Linhas pontilhadas para fechar o paralelogramo
plt.plot([0, step1[0]], [0, step1[1]], 'b--', alpha=0.4, label=f'Componente b1
({c1:.1f} passos)')
plt.plot([step1[0], v[0]], [step1[1], v[1]], 'g--', alpha=0.4,
label=f'Componente b2 ({c2:.1f} passos)')

plt.xlim(-2, 6)
plt.ylim(-1, 6)
plt.grid(True, linestyle=':', alpha=0.6)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
```

```
plt.title(f"Coordenadas de v na Nova Base:\n v = {c1:.1f} * b1 + {c2:.1f} * b2")
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```

vii. Mudança de Base

Nos capítulos anteriores, vimos que:

- uma **base** fornece um conjunto de direções fundamentais do espaço;
- as **coordenadas** de um vetor dependem da base escolhida;
- um mesmo vetor físico pode ter representações diferentes.

Agora damos um passo crucial:

como converter coordenadas de uma base para outra.

A mudança de base é um dos conceitos mais importantes da Álgebra Linear aplicada à Computação, aparecendo diretamente em Análise de Componentes Principais (PCA), compressão, gráficos, visão computacional e redes neurais.

1. Intuição: O vetor é o mesmo, a descrição muda

Imagine que temos um endereço em um mapa feito com:

- Norte–Sul
- Leste–Oeste

Esse é o sistema de coordenadas padrão (base canônica).

Se trocarmos para um mapa inclinado, por exemplo:

- “direção 1”: diagonal para nordeste “/”
- “direção 2”: diagonal para noroeste “\”

O local físico não muda. Mas o **endereço numérico muda completamente**.

Assim ocorre com vetores:

- O vetor permanece o mesmo ponto no espaço.
- O que muda são suas coordenadas quando mudamos a base.

2. Definição Formal da Mudança de Base

Seja:

- $\mathcal{B} = \{b_1, b_2, \dots, b_n\}$ a primeira base;
- $\mathcal{C} = \{c_1, c_2, \dots, c_n\}$ a nova base.

Um vetor v pode ser escrito:

- na base $\mathcal{B}$:

$$v = a_1b_1 + \cdots + a_nb_n \quad \Rightarrow \quad [v]_B = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

- na base $\mathcal{C}$:

$$v = d_1c_1 + \cdots + d_nc_n \quad \Rightarrow \quad [v]_{\mathcal{C}} = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_n \end{bmatrix}$$

O vetor é o mesmo — o que muda são os coeficientes.

3. Exemplo Completo

Vamos converter o vetor

$$v = (4,3)$$

da base canônica para uma nova base:

$$\mathcal{C} = \{c_1 = (2,1), c_2 = (-1,1)\}$$

A pergunta é:

Quais são os números d_1 e d_2 tais que

$$v = d_1c_1 + d_2c_2$$

Passo 1: Escrever a igualdade vetorial explicitamente

Substituímos os vetores:

$$(4,3) = d_1(2,1) + d_2(-1,1)$$

Agora distribuímos os escalares:

$$(4,3) = (2d_1, d_1) + (-d_2, d_2)$$

Passo 2: Somar componente a componente

Somando os dois vetores da direita:

$$(4,3) = (2d_1 - d_2, d_1 + d_2)$$

Essa igualdade de vetores significa:

- a **primeira componente** da esquerda deve igualar a **primeira componente** da direita;
- a mesma coisa para a segunda.

Então obtemos um sistema de equações:

$$\begin{cases} 2d_1 - d_2 = 4 \\ d_1 + d_2 = 3 \end{cases}$$

Passo 3: Resolver o sistema

Somamos as duas equações para eliminar d_2 :

$$(2d_1 - d_2) + (d_1 + d_2) = 4 + 3$$

$$3d_1 = 7 \quad \Rightarrow \quad d_1 = \frac{7}{3}$$

Agora substituímos em uma das equações:

$$d_1 + d_2 = 3$$

$$\frac{7}{3} + d_2 = 3$$

$$d_2 = 3 - \frac{7}{3}$$

$$d_2 = \frac{9}{3} - \frac{7}{3} = \frac{2}{3}$$

Passo 4: Conclusão: coordenadas de v na nova base

$$[v]_C = \begin{bmatrix} d_1 \\ d_2 \end{bmatrix} = \begin{bmatrix} \frac{7}{3} \\ \frac{2}{3} \end{bmatrix}$$

ou seja, o vetor $v = (4,3)$ é formado por $\frac{7}{3}$ passos na direção de c_1 e $\frac{2}{3}$ passos na direção de c_2 .

Passo 5: Verificação (para reforçar o entendimento)

Vamos reconstruir o vetor com os coeficientes encontrados:

$$d_1(2,1) + d_2(-1,1) = \frac{7}{3}(2,1) + \frac{2}{3}(-1,1)$$

Distribuindo:

$$= \left(\frac{14}{3}, \frac{7}{3}\right) + \left(\frac{-2}{3}, \frac{2}{3}\right)$$

Somando:

$$= \left(\frac{12}{3}, \frac{9}{3}\right) = (4,3)$$

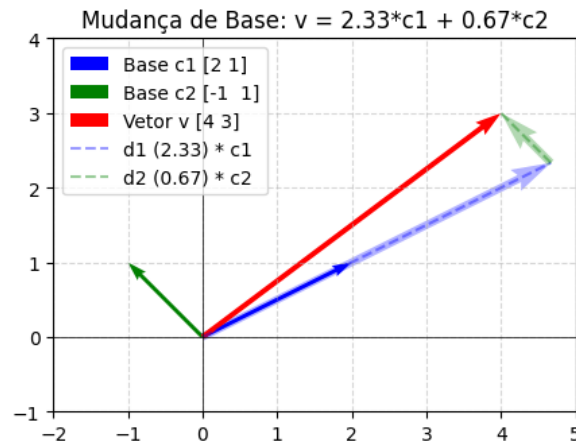
O resultado confere perfeitamente.

Passo 6: Interpretação geométrica

- O vetor v continua sendo o mesmo ponto no plano.
- O que muda são as **instruções de navegação** no novo sistema de eixos:
 - $\frac{7}{3}$ passos na direção de c_1
 - $\frac{2}{3}$ passos na direção de c_2

É assim que se obtém coordenadas na nova base.

Passo 8: Implementação em Python



```
import numpy as np
import matplotlib.pyplot as plt

# 1. Definição do Vetor e da Nova Base (conforme o texto)
v = np.array([4, 3])    # Vetor no sistema padrão
c1 = np.array([2, 1])   # Primeiro vetor da nova base
c2 = np.array([-1, 1])  # Segundo vetor da nova base

# 2. Montagem da Matriz de Mudança de Base
# A matriz P_C deve ter os vetores da base C como COLUNAS
#P_C = [[2, -1], [1, 1]]
P_C = np.column_stack((c1, c2))

# 3. Resolução do Sistema Linear
# Queremos encontrar d (vetor de coordenadas novas) tal que: P_C * d = v
d = np.linalg.solve(P_C, v)
d1, d2 = d

# Exibição dos resultados numéricos
print("=== Resultados da Mudança de Base ===")
print(f"Vetor Original (v): {v}")
print(f"Matriz da Base C (colunas):\n{P_C}")
print(f"\nSolução encontrada (Coordenadas na Base C):")
print(f"d1 (peso de c1): {d1:.4f} (Fração exata: 7/3)")
print(f"d2 (peso de c2): {d2:.4f} (Fração exata: 2/3)")

# Verificação: Reconstruindo o vetor
v_reconstruido = d1 * c1 + d2 * c2
print(f"\nVerificação (d1*c1 + d2*c2): {v_reconstruido}")

# 4. Visualização Gráfica
plt.figure(figsize=(5, 5))
origin = [0, 0]

# a) Plotar os vetores da Nova Base (para referência)
plt.quiver(*origin, c1[0], c1[1], color='blue', scale=1, scale_units='xy',
angles='xy', label=f'Base c1 {c1}')
```

```
plt.quiver(*origin, c2[0], c2[1], color='green', scale=1, scale_units='xy',
angles='xy', label=f'Base c2 {c2}')

# b) Plotar o "Caminho" usando as novas coordenadas
# Passo 1: Andar d1 vezes na direção de c1
passo1 = d1 * c1
plt.quiver(*origin, passo1[0], passo1[1], color='blue', scale=1,
scale_units='xy', angles='xy', alpha=0.3, width=0.015)

# Passo 2: A partir de onde paramos, andar d2 vezes na direção de c2
passo2 = d2 * c2
plt.quiver(passo1[0], passo1[1], passo2[0], passo2[1], color='green', scale=1,
scale_units='xy', angles='xy', alpha=0.3, width=0.015)

# c) Plotar o Vetor Original (Destino)
plt.quiver(*origin, v[0], v[1], color='red', scale=1, scale_units='xy',
angles='xy', label=f'Vetor v {v}', width=0.010)

# Linhas pontilhadas para fechar o paralelogramo visualmente
plt.plot([0, passo1[0]], [0, passo1[1]], 'b--', alpha=0.4, label=f'd1 ({d1:.2f})
* c1')
plt.plot([passo1[0], v[0]], [passo1[1], v[1]], 'g--', alpha=0.4, label=f'd2
({d2:.2f}) * c2')

# Ajustes do Gráfico
plt.xlim(-2, 5)
plt.ylim(-1, 4)
plt.grid(True, linestyle='--', alpha=0.5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend(loc='upper left')
plt.title(f'Mudança de Base: v = {d1:.2f}*c1 + {d2:.2f}*c2')
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```

Exercícios

Combinação Linear – Exercícios 1 e 2

Exercício 1 - Considere os vetores $v_1 = (1,2)$ e $v_2 = (3,1)$.

Calcule explicitamente as seguintes combinações lineares:

- a) $-3v_1 + 4v_2$
- b) $0,5 v_1 + 1,5 v_2$

Resposta:

- a) $-3v_1 + 4v_2 = -3 \cdot (1,2) + 4 \cdot (3,1) = (-3, -6) + (12,4) = (9, -2)$
- b) $0,5 v_1 + 1,5 v_2 = 0,5 \cdot (1,2) + 1,5 \cdot (3,1) = (0,5,1) + (4,5,1,5) = (5,2,5)$

Exercício 2 - Considere os vetores $v_1 = (1,2)$ e $v_2 = (3,1)$.

Dado o vetor $w = (7,4)$, encontre escalares α, β tais que

$$w = \alpha v_1 + \beta v_2$$

Resposta:

Queremos

$$(7,4) = \alpha(1,2) + \beta(3,1)$$

1. Combinação linear componente a componente:

$$\alpha(1,2) = (\alpha, 2\alpha), \quad \beta(3,1) = (3\beta, \beta)$$

Somando:

$$(\alpha + 3\beta, 2\alpha + \beta) = (7,4)$$

2. Igualando componentes, obtemos o sistema:

$$\begin{cases} \alpha + 3\beta = 7 & (1) \\ 2\alpha + \beta = 4 & (2) \end{cases}$$

3. Da equação (1):

$$\alpha = 7 - 3\beta$$

4. Substitua em (2):

$$2(7 - 3\beta) + \beta = 4$$

$$14 - 6\beta + \beta = 4$$

$$5\beta = 4 - 14 = -10$$

$$\beta = 2$$

5. Volte em $\alpha = 7 - 3\beta$:

$$\alpha = 7 - 3 \cdot 2 = 7 - 6 = 1$$

Resposta: $w = 1 \cdot v_1 + 2 \cdot v_2$

Conjunto Gerador (Span) – Exercício 3

Exercício 3 - Considere

$$v_1 = (1,2), \quad v_2 = (-1,1)$$

Mostre que o span de $\{v_1, v_2\}$ é igual a R^2 .

Dica: verifique se existe combinação linear que produza $(1,0)$ e $(0,1)$.

Resposta:

Uma forma padrão de provar que o span é R^2 é mostrar que podemos obter $(1,0)$ e $(0,1)$ como combinações lineares de v_1, v_2 .

Passo 1: Obter $(1,0)$

Procuramos α, β tais que:

$$\alpha(1,2) + \beta(-1,1) = (1,0)$$

Componentes:

$$(\alpha - \beta, 2\alpha + \beta) = (1, 0)$$

Sistema:

$$\begin{cases} \alpha - \beta = 1 & (1) \\ 2\alpha + \beta = 0 & (2) \end{cases}$$

De (2):

$$\beta = -2\alpha$$

Substitua em (1):

$$\alpha - (-2\alpha) = 1 \Rightarrow 3\alpha = 1 \Rightarrow \alpha = \frac{1}{3}$$

Então:

$$\beta = -2 \cdot \frac{1}{3} = -\frac{2}{3}$$

Logo:

$$(1, 0) = \frac{1}{3}v_1 - \frac{2}{3}v_2$$

Passo 2: Obter (0,1)

Agora, γ, δ tais que:

$$\gamma(1, 2) + \delta(-1, 1) = (0, 1)$$

Componentes:

$$(\gamma - \delta, 2\gamma + \delta) = (0, 1)$$

Sistema:

$$\begin{cases} \gamma - \delta = 0 & (1') \\ 2\gamma + \delta = 1 & (2') \end{cases}$$

De (1'):

$$\gamma = \delta$$

Substitua em (2'):

$$2\gamma + \gamma = 1 \Rightarrow 3\gamma = 1 \Rightarrow \gamma = \frac{1}{3}$$

Logo $\delta = \gamma = \frac{1}{3}$.

Portanto:

$$(0, 1) = \frac{1}{3}v_1 + \frac{1}{3}v_2$$

Como obtemos (1,0) e (0,1) a partir de v_1 e v_2 , e esses vetores geram R^2 , concluímos:

$$\text{span}\{v_1, v_2\} = R^2$$

Dependência e Independência Linear – Exercício 4 e 5

Exercício 4 -Determine se o conjunto é LI ou LD:

$$\{(2,1), (4,2)\}$$

Justifique pela definição formal.

Resposta:

Considere a combinação:

$$\alpha(2,1) + \beta(4,2) = (0,0)$$

Componentes:

$$(2\alpha + 4\beta, \alpha + 2\beta) = (0,0)$$

Sistema:

$$\begin{cases} 2\alpha + 4\beta = 0 & (1) \\ \alpha + 2\beta = 0 & (2) \end{cases}$$

Note que (1) é apenas $2 \times (2)$. Logo as duas equações são equivalentes.

De (2):

$$\alpha = -2\beta$$

Se escolhermos $\beta = 1$, então $\alpha = -2$. Temos uma solução **não trivial** ($\alpha, \beta \neq 0$):

$$-2(2,1) + 1(4,2) = (0,0)$$

Como existe combinação linear não trivial que dá o vetor nulo, o conjunto é LD.

Exercício 5 - Mostre que o vetor $v_3 = (2,3)$ é LD em relação ao conjunto

$$\{v_1 = (2,1), v_2 = (-1,1)\}$$

Encontre coeficientes que satisfaçam $v_3 = av_1 + bv_2$.

Resposta:

Queremos:

$$a(2,1) + b(-1,1) = (2,3)$$

Componentes:

$$(2a - b, a + b) = (2,3)$$

Sistema:

$$\begin{cases} 2a - b & (1) \\ a + b & (2) \end{cases}$$

De (2):

$$b = 3 - a$$

Substitua em (1):

$$2a - (3 - a) = 2$$

$$2a - 3 + a = 2$$

$$3a - 3 = 2$$

$$3a = 5 \Rightarrow a = \frac{5}{3}$$

Então:

$$b = 3 - \frac{5}{3} = \frac{9-5}{3} = \frac{4}{3}$$

Logo:

$$v_3 = \frac{5}{3}v_1 + \frac{4}{3}v_2$$

Como v_3 é combinação linear de v_1 e v_2 , ele é dependente em relação a esse conjunto.

Bases Vetoriais– Exercício 6

Exercício 6 - Explique por que o conjunto

$$C = \{(1,1), (2,2), (3,3)\}$$

não pode ser uma base.

Resposta:

Observe que:

$$(2,2) = 2(1,1), \quad (3,3) = 3(1,1)$$

Isso significa que todos os vetores são múltiplos de $(1,1)$, ou seja, todos estão na mesma reta.

Assim, o span de C é apenas uma reta em R^2 , não o plano todo.

Além disso, uma base de R^2 deve ter exatamente 2 vetores LI. Com 3 vetores em R^2 , é impossível que sejam todos LI (sempre haverá dependência).

Logo,

$$C \text{ não é base de } R^2$$

Coordenadas em uma Base – Exercícios 7

Exercício 7- Considere a base

$$B = \{b_1 = (2,1), b_2 = (-1,1)\}$$

Encontre as coordenadas na base B do vetor

$$v = (5,2)$$

Resposta:

Queremos (c_1, c_2) tais que:

$$v = c_1b_1 + c_2b_2$$

Isto é:

$$c_1(2,1) + c_2(-1,1) = (5,2)$$

Componentes:

$$(2c_1 - c_2, c_1 + c_2) = (5,2)$$

Sistema:

$$\begin{cases} 2c_1 - c_2 & (1) \\ c_1 + c_2 & (2) \end{cases}$$

De (2):

$$c_2 = 2 - c_1$$

Substitua em (1):

$$2c_1 - (2 - c_1) = 5$$

$$2c_1 - 2 + c_1 = 5$$

$$3c_1 - 2 = 5$$

$$3c_1 = 7 \Rightarrow c_1 = \frac{7}{3}$$

Então:

$$c_2 = 2 - \frac{7}{3} = \frac{6-7}{3} = -\frac{1}{3}$$

Verificação rápida:

$$\frac{7}{3}(2,1) - \frac{1}{3}(-1,1) = \left(\frac{14}{3}, \frac{7}{3}\right) + \left(\frac{1}{3}, -\frac{1}{3}\right) = \left(\frac{15}{3}, \frac{6}{3}\right) = (5,2)$$

Logo, as coordenadas de v na base B são:

$$[v]_B = \begin{pmatrix} 7/3 \\ -1/3 \end{pmatrix}$$

Mudança de Base – Exercícios 8 e 9

Exercício 8 – Com a base $C = \{c_1 = (2,1), c_2 = (-1,1)\}$ se $[v]_C = (1, -3)$ encontre v em coordenadas padrão.

Resposta:

Por definição:

$$v = 1 \cdot c_1 + (-3) \cdot c_2 = c_1 - 3c_2$$

Calcule $-3c_2$:

$$-3(-1,1) = (3, -3)$$

Some com c_1 :

$$v = (2,1) + (3, -3) = (5, -2)$$

Resposta: $v = (5, -2)$.

Exercício 9 - Suponha que um algoritmo de redução de dimensionalidade projete um vetor 2D na base

$$u_1 = (1,1), \quad u_2 = (1, -1)$$

obtendo coordenadas $(8, -2)$.

- Reconstrua o vetor original.
- Interprete geometricamente o efeito da mudança de base.

Resposta:

a) Reconstrução de v

Temos:

$$v = 8u_1 + (-2)u_2 = 8u_1 - 2u_2$$

Calcule $8u_1$:

$$8u_1 = 8(1,1) = (8,8)$$

Calcule $-2u_2$:

$$-2u_2 = -2(1, -1) = (-2,2)$$

Some:

$$v = (8,8) + (-2,2) = (8 - 2, 8 + 2) = (6,10)$$

Resposta da parte (a):

$$v = (6,10)$$

b) Interpretação geométrica

Os vetores $u_1 = (1,1)$ e $u_2 = (1, -1)$ são direções “inclinadas” em relação à base padrão;

- u_1 : diagonal que sobe (nordeste);
- u_2 : diagonal que desce (sudeste/noroeste, dependendo do sentido).

Escrever o vetor em termos de u_1 e u_2 é como usar um sistema de eixos rotacionado em 45° .